

Experiment:7

Construct a C program to implement non-preemptive SJF algorithm

Code:-

```
#include <stdio.h>

int main()
{
    int at[10], bt[10], pr[10];
    int n, i, j, temp;
    int time = 0, count, over = 0;
    int sum_wait = 0, sum_turnaround = 0, start;
    float avgwait, avgturn;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("Enter Arrival Time and Burst Time for Process %d:\n", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        pr[i] = i + 1;
    }
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (at[i] > at[j])
            {
```

```

        temp = at[i];
        at[i] = at[j];
        at[j] = temp;

        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;

        temp = pr[i];
        pr[i] = pr[j];
        pr[j] = temp;
    }
}

}

printf("\nProcess\tAT\tBT\tST\tET\tWT\tTAT\n");

while (over < n)
{
    count = 0;
    for (i = over; i < n; i++)
    {
        if (at[i] <= time)
            count++;
        else
            break;
    }
    if (count > 1)
    {
        for (i = over; i < over + count - 1; i++)

```

```

    {
        for (j = i + 1; j < over + count; j++)
        {
            if (bt[i] > bt[j])
            {
                temp = at[i];
                at[i] = at[j];
                at[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
            }
        }
    }
}

start = time;

time += bt[over];

printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pr[over],
        at[over],
        bt[over],
        start,

```

```

        time,
        time - at[over] - bt[over],
        time - at[over]));
sum_wait += time - at[over] - bt[over];
    sum_turnaround += time - at[over];

    over++;
}
avgwait = (float)sum_wait / n;
avgturn = (float)sum_turnaround / n;
printf("\nAverage Waiting Time = %.2f", avgwait);
printf("\nAverage Turnaround Time = %.2f\n", avgturn);

return 0;
}

```

Output:

```

Enter the number of processes: 3
Enter Arrival Time and Burst Time for Process 1:
2
4
Enter Arrival Time and Burst Time for Process 2:
2
4
Enter Arrival Time and Burst Time for Process 3:
82
4

Process AT      BT      ST      ET      WT      TAT
P1      2       4       0       4       -2       2
P2      2       4       4       8       2       6
P3     82       4       8      12     -74     -70

Average Waiting Time = -24.67
Average Turnaround Time = -20.67

...Program finished with exit code 0
Press ENTER to exit console.

```

